

Relatório Técnico - Parte Prática

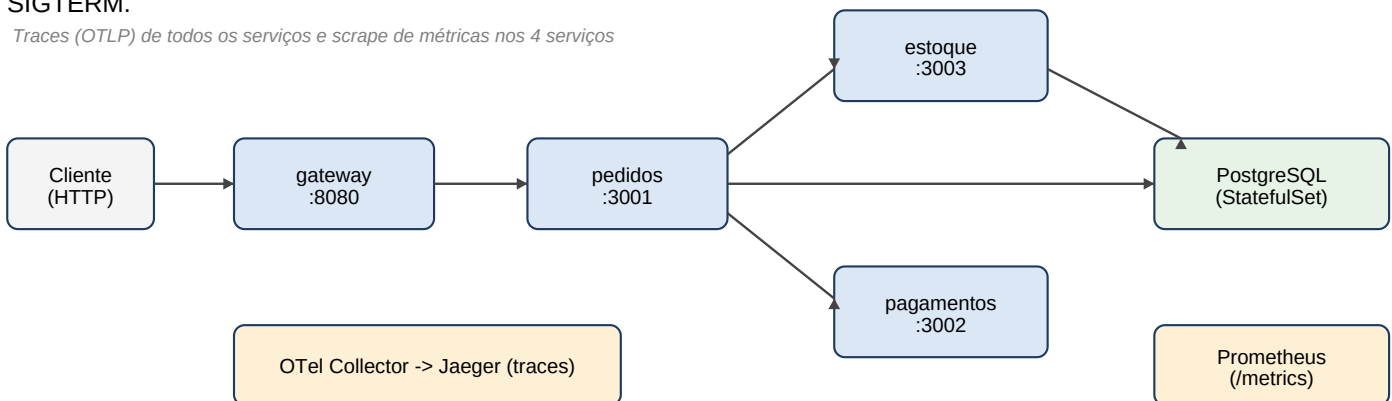
Entrega contínua de uma plataforma de pedidos em microsserviços: do Docker Compose ao Kubernetes com observabilidade e CI/CD

Aluno: Francisco Guilherme do Nascimento Costa | RA: 128984 | Disciplina: Cloud DevOps - Orchestrating Containers and Micro Services | Tema: Plataforma de pedidos em microsserviços (Loja Veloz)
Repositório: <https://github.com/guifrancosta75/loja-veloz-devops>

1. Visão geral e arquitetura

A solução entrega o caminho completo do desenvolvimento à produção para o “Pedidos Veloz”: quatro microsserviços em Node.js (**gateway**, **pedidos**, **pagamentos**, **estoque**) e PostgreSQL. O serviço de pedidos orquestra o fluxo: reserva estoque (transação atômica), solicita o pagamento e grava o pedido; se o pagamento é recusado, executa uma **compensação** (libera o estoque), padrão *saga*. Todos expõem `/healthz`, `/readyz` e `/metrics`, escrevem logs JSON em stdout e encerram graciosamente ao receber SIGTERM.

Traces (OTLP) de todos os serviços e scrape de métricas nos 4 serviços



2. Case de referência pesquisado

Como exemplo concreto foi estudado o **Online Boutique** (GoogleCloudPlatform/microservices-demo), aplicação e-commerce de referência do Google, composta por cerca de 10 microsserviços, usada para demonstrar Kubernetes/GKE, Istio e gRPC. Seu repositório oficial mostra práticas que adotei: manifests Kubernetes versionados e implantáveis com um comando, uso de Kustomize para variantes, módulos Terraform para criar o cluster, gerador de carga sintética e integração opcional com service mesh e observabilidade. **Adaptações:** em vez de gRPC poliglota, usei HTTP/JSON em uma única stack para reduzir o custo e a complexidade do MVP; adicionei Compose como ambiente local, NetworkPolicies, Pod Security «restricted», pipeline com scan de vulnerabilidades e rollback, e tracing com OpenTelemetry.

3. Ambiente local com Docker Compose

O arquivo `compose.yaml` define 8 containers (4 serviços, PostgreSQL, OpenTelemetry Collector, Jaeger e Prometheus) e sobe com **um único comando:** `docker compose up --build -d`. **Redes:** *frontend* (apenas o gateway, que publica a porta 8080) e *backend* (interna, sem exposição ao host). **Volume:** *pgdata* persiste o banco; o esquema e dados de exemplo vêm de `sql/init.sql`. **Variáveis:** senhas e chaves vêm de `.env` (não versionado; há `.env.example`). O PostgreSQL tem *healthcheck* e os serviços dependentes aguardam que fique saudável. O README traz as instruções, o script `scripts/smoke.sh` testa o fluxo ponta a ponta e a variável `PAGAMENTO_FALHA_PERCENT` permite simular falhas de pagamento.

4. Containerização e versionamento

Um único Dockerfile *multi-stage* gera as quatro imagens (`--build-arg SERVICE`), evitando duplicação. O primeiro estágio instala apenas dependências de produção (`npm ci --omit=dev`); o final copia só o necessário sobre `node:20-alpine`.

- **Segurança:** execução como usuário não-root (`USER node`), base Alpine mínima, sem cache do npm, `HEALTHCHECK`, `.dockerignore` e `exec` para o processo ser PID 1 e receber SIGTERM.
- **Versionamento:** cada imagem recebe tags `sha-<commit>` (imutável, usada no deploy), `x.y.z` (tags Git `vX.Y.Z`) e `latest`, além de labels OCI com a versão. Registry: GitHub Container Registry. O deploy nunca usa `latest`, garantindo rastreabilidade e rollback.

5. Kubernetes - produção mínima

Os manifests estão organizados com **Kustomize** (k8s/base + overlays *local* e *prod*). Para cada serviço: Deployment, Service, ServiceAccount, PodDisruptionBudget e HPA.

- **Configuração:** ConfigMap gerado pelo Kustomize (URLs internas, endpoint OTLP); o hash no nome força novo rollout quando muda. **Secrets** (senha do banco, DATABASE_URL, chave do provedor de pagamento) não ficam no Git: são criados pelo pipeline a partir do cofre do GitHub (há apenas um *secrets.example.yaml*); em evolução, External Secrets/Vault.
- **Probes:** *startup*, *readiness* (/readyz, que verifica o banco; retira o Pod do balanceamento) e *liveness* (/healthz; reinicia). *preStop*: sleep 5 + terminationGracePeriodSeconds drenam conexões antes de encerrar.
- **Segurança (justificada):** namespace com **Pod Security Admission «restricted»** (rejeita Pods privilegiados ou root, prática nativa e sem componente extra); securityContext com runAsNonRoot, filesystem raiz somente leitura (/tmp em emptyDir), capabilities drop ALL, sem escalonamento de privilégio e seccomp RuntimeDefault; token da ServiceAccount desabilitado; **NetworkPolicy** com *default-deny* e regras mínimas (Ingress -> gateway -> pedidos -> estoque/pagamentos -> Postgres) para conter movimento lateral; requests/limits de CPU e memória.

6. Pipeline CI/CD (GitHub Actions)

O arquivo .github/workflows/ci-cd.yml executa quatro estágios encadeados:

Job	O que faz	Gate
test	npm ci (com cache), lint, testes unitários	Falha bloqueia o restante
validate	hadolint (Dockerfile), kustomize + kubeconform (manifests), terraform validate, checagem de sincronia do init.sql	Erro de configuração é pego antes do deploy
build-push	Matriz de 4 serviços: build com cache, scan Trivy (falha em CRITICAL corrigível), login e push ao GHCR com tags sha/semver/latest	Push só fora de Pull Requests
deploy	Cria o Secret a partir de GitHub Secrets, fixa a tag sha nas imagens (kustomize), aplica, aguarda rollout status	Ambiente <i>production</i> (aprovação manual) e rollback automático

Uso de secrets: KUBE_CONFIG_B64, POSTGRES_PASSWORD e PAYMENT_PROVIDER_API_KEY ficam no GitHub Secrets; o push usa o GITHUB_TOKEN com permissões mínimas (permissions declaradas por job).

7. Observabilidade, deploy e escala

Métricas. prom-client expõe métricas de runtime e o histograma http_request_duration_seconds (rótulos método, rota, status), base dos *golden signals* (taxa, erros, latência). Em Compose, o Prometheus coleta os 4 serviços; no cluster, *kube-prometheus-stack* com ServiceMonitor e regras de alerta (erros 5xx > 5% por 5 min; p95 > 1 s por 10 min). **Logs.** JSON em stdout (12-Factor XI) com trace_id em cada linha, permitindo correlacionar log e trace; coleta por Loki/Promtail ou equivalente é a evolução. **Tracing distribuído (instrumentado).** OpenTelemetry (HTTP, Express, pg e fetch) propaga o cabeçalho *traceparent* entre gateway, pedidos, estoque e pagamentos e exporta via OTLP ao Collector e ao Jaeger; assim um pedido lento aparece como um único trace com spans por serviço e por consulta SQL. Os probes e o /metrics são ignorados no tracing para reduzir ruído.

Estratégia de deploy: Rolling Update (maxSurge 1, maxUnavailable 0, minReadySeconds 10), apoiado por readiness probe e rollback automático no pipeline. Justificativa: atende o problema central (indisponibilidade em deploys) sem duplicar o ambiente (blue/green) nem exigir malha de serviço (canary por pesos). Como evolução, o serviço de pagamentos – o mais crítico e dependente de terceiro – pode usar **canary** com Istio (exemplo em k8s/ext ras: 90/10, com mTLS STRICT).

Escalabilidade: HPA por CPU (alvo 70%) – gateway e pedidos de 2 a 10 réplicas, estoque e pagamentos de 2 a 6. Scale-up rápido (até +100% a cada 30 s) para absorver o pico e scale-down com janela de 5 min para evitar oscilação. Mínimo de 2 réplicas com PDB e *topology spread* garante disponibilidade durante manutenções. O **VPA** seria usado só em modo de recomendação para calibrar requests/limits, pois o modo automático reinicia Pods e conflita com o HPA baseado em CPU. Requer *metrics-server* e, para escalar nós, Cluster Autoscaler/Karpenter.

8. Infraestrutura como código (Terraform)

O diretório `infra/terraform` contém um esqueleto com módulos oficiais *terraform-aws-modules*: VPC em 3 zonas (sub-redes públicas/privadas, NAT) e cluster EKS com *managed node group* elástico e add-ons essenciais. Variáveis e arquivos `envs/dev.tfvars` e `prod.tfvars` parametrizam os ambientes (p. ex., um NAT por zona só em produção); o backend S3 com lock em DynamoDB está preparado (comentado) para estado remoto seguro. Justificativa: cluster reproduzível e revisável por *pull request*, com as mesmas boas práticas de versionamento do código.

9. Evidências de validação e limitações

Executado e verificado: lint (`node --check`) e 6 testes unitários aprovados (health/metrics nos 4 serviços, readiness com banco fora do ar, pagamento, reserva com 409, fluxo de pedido com e sem compensação, gateway com 504); inicialização de serviço com tracing ativo, exibindo `trace_id` nos logs e encerramento por SIGTERM; e análise sintática e estrutural dos 45 documentos YAML (strategy, probes, resources e securityContext presentes em todos os Deployments). **Não executado neste ambiente:** docker compose up, cluster Kubernetes real, pipeline no GitHub e terraform apply. **Trabalhos futuros:** banco gerenciado em produção, mensageria para o evento PedidoCriado, service mesh com canary, GitOps (Argo CD), coleta central de logs e testes de carga para calibrar o HPA.

Referências

- Google Cloud Platform. *Online Boutique - microservices-demo*. <https://github.com/GoogleCloudPlatform/microservices-demo>
- Kubernetes Documentation (Deployments, Probes, HPA, Secrets, Pod Security Standards, NetworkPolicy). <https://kubernetes.io/docs/>
- Docker Documentation (Dockerfile, multi-stage, Compose). <https://docs.docker.com/> | The Twelve-Factor App. <https://12factor.net/>
- HashiCorp Terraform Documentation. <https://developer.hashicorp.com/terraform/docs> | GitHub Actions Docs. <https://docs.github.com/actions>
- OpenTelemetry Documentation. <https://opentelemetry.io/docs/> | Istio Documentation. <https://istio.io/latest/docs/>